

# FALCON

Fast Adaptive Localization via Correlation & Kalman-filter Observation Network

Signal\_13

September 25, 2026

# Why the name FALCON?

- A falcon locks onto fast-moving prey, tracks it in flight, and re-acquires it if it briefly disappears behind an obstacle.
- That is exactly the behaviour we engineered: **fast local tracking** while the motion is predictable, and **automatic re-acquisition** when it is not.

# Why the name FALCON?

- A falcon locks onto fast-moving prey, tracks it in flight, and re-acquires it if it briefly disappears behind an obstacle.
- That is exactly the behaviour we engineered: **fast local tracking** while the motion is predictable, and **automatic re-acquisition** when it is not.
- So we named the project after the bird — and built the acronym around what it actually does.

**F**

**Fast**

Search only a small local region, not the whole frame

# FALCON — Letter by Letter

**F**    **Fast**                      Search only a small local region, not the whole frame

**A**    **Adaptive**                      Working template updates slowly to small appearance change

# FALCON — Letter by Letter

**F**    **Fast**                      Search only a small local region, not the whole frame

**A**    **Adaptive**                    Working template updates slowly to small appearance change

**L**    **Localization**                Finding *where* the target is, frame to frame

# FALCON — Letter by Letter

**F**    **Fast**    Search only a small local region, not the whole frame

**A**    **Adaptive**    Working template updates slowly to small appearance change

**L**    **Localization**    Finding *where* the target is, frame to frame

**C**    **Correlation**    Custom FFT phase correlation finds the displacement

# FALCON — Letter by Letter

**F**    **Fast**    Search only a small local region, not the whole frame

**A**    **Adaptive**    Working template updates slowly to small appearance change

**L**    **Localization**    Finding *where* the target is, frame to frame

**C**    **Correlation**    Custom FFT phase correlation finds the displacement

**O**    **Observation**    PSR + NCC “sanity checks” on every candidate match

# FALCON — Letter by Letter

**F**    **Fast**                      Search only a small local region, not the whole frame

**A**    **Adaptive**                    Working template updates slowly to small appearance change

**L**    **Localization**                Finding *where* the target is, frame to frame

**C**    **Correlation**                  Custom FFT phase correlation finds the displacement

**O**    **Observation**                PSR + NCC “sanity checks” on every candidate match

**N**    **Network**                      Full pipeline: Django app, DSP modules, analytics

# FALCON — Letter by Letter

**F**    **Fast**                      Search only a small local region, not the whole frame

**A**    **Adaptive**                    Working template updates slowly to small appearance change

**L**    **Localization**                Finding *where* the target is, frame to frame

**C**    **Correlation**                  Custom FFT phase correlation finds the displacement

**O**    **Observation**                PSR + NCC “sanity checks” on every candidate match

**N**    **Network**                      Full pipeline: Django app, DSP modules, analytics

*Fast Adaptive Localization via Correlation & Kalman-filter Observation Network*

## FALCON in One Paragraph

A local, browser-based object tracker. The user selects a target (from the video, or from a separate reference image). FALCON tracks it using custom FFT phase correlation, checks every match with PSR and NCC, and uses a Kalman filter to predict motion. If the target is occluded or jumps away, the system switches to a time-sliced global search until it is found again.

# Motivation and Problem Addressed

- Many simple trackers work only while the object stays near the previous position.
- In our test cases, the object can be occluded, jump to another place, or appear later from a reference image.
- Searching the full frame every time is possible, but it is too slow for a CPU-based custom FFT pipeline.
- **So the project problem is:** track locally when possible, but re-acquire globally when local tracking fails.

# The Core Problems

- ❶ **Tracking efficiency:** Real-time tracking requires CPU-efficient processing. Searching the entire frame for a small target every frame is wasteful.
- ❷ **Noisy correspondence:** Phase correlation always returns a peak, even on background or noise. How do we know when to trust it?
- ❸ **Handling disappearance:** Targets get occluded, jump away, or may not appear until later in the video. A tracker must survive these gaps.
- ❹ **Stability under drift:** Appearance changes accumulate over time. The tracker must not drift away from the true target.

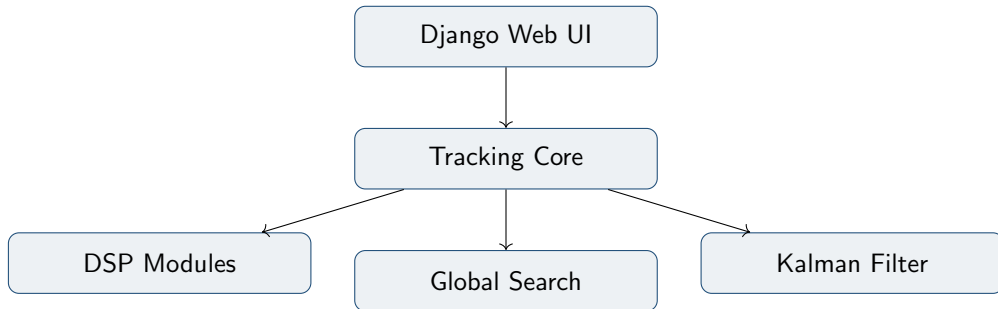
## Our Solution: Layered Integration

| Problem                          | FALCON's Answer                      |
|----------------------------------|--------------------------------------|
| Expensive global search          | Kalman prediction + local FFT        |
| Distinguishing noise from signal | PSR + NCC                            |
| Lost or occluded targets         | Time-sliced global grid search       |
| Template drift                   | Fixed reference anchor + slow update |

*No single technique is sufficient — the combination is robust.*

# Important Implementation Decisions

- Use a **local search window** during normal tracking to keep FFT size small.
- Use **overlapping global tiles** only when local tracking fails.
- Keep a **fixed original reference template** for re-acquisition, while allowing a slow working-template update during normal tracking.
- Reset Kalman position and velocity after global re-acquisition instead of trying to predict a jump.



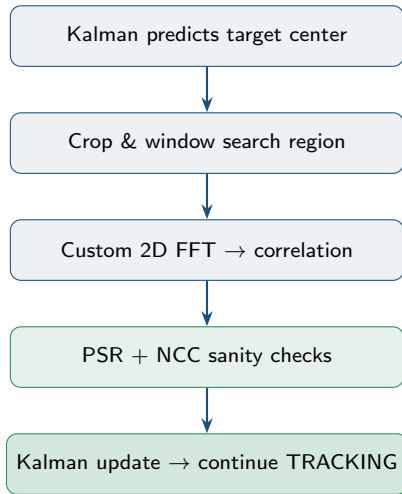
## Theory: Phase Correlation in Brief

- Take the FFT of the search patch and template.
- Keep only their phase difference using the cross-power spectrum:

$$R(u, v) = \frac{F(u, v) T^*(u, v)}{|F(u, v) T^*(u, v)|}, \quad r(x, y) = \mathcal{F}^{-1}\{R(u, v)\}$$

- The highest peak of  $r(x, y)$  gives the estimated shift  $(\Delta x, \Delta y)$ .
- This is why we can estimate motion without checking every pixel position manually.

## Local Tracking — Frame-by-Frame Flow



# Why Each Piece Is Needed

| Component          | Solves                                                |
|--------------------|-------------------------------------------------------|
| Phase correlation  | "How far has the pattern moved?"                      |
| PSR                | "Is this peak actually convincing, or just noise?"    |
| NCC                | "Does the matched patch really look like the target?" |
| Kalman filter      | "Where should I even look next?"                      |
| Global grid search | "What if it's nowhere near the prediction?"           |

*No single piece is sufficient alone — the combination is the contribution.*

# Our Approach: Constant Velocity Model

- We assume **constant velocity** motion — no acceleration model needed.
- Two phases, each frame:
  - ① **Predict:** “Where should the target be, based on its speed?” → centers the local search window.
  - ② **Update:** If phase correlation + PSR/NCC pass, correct the velocity estimate using that measurement.
- If local tracking fails, velocity is **reset** when we re-acquire the target globally — we don't try to guess teleports or occlusions.

**Honest scope:** Kalman models *ordinary continuous motion*. It is not designed to predict occlusion or large jumps.

# Why We Don't Trust the FFT Peak Blindly

- Phase correlation always returns *some* peak — even on pure background or noise.
- We need two independent “sanity checks” before we trust a match:
  - Is the peak actually **distinctive**? → **PSR**
  - Does the matched region **look like** our target? → **NCC**

# PSR — Peak-to-Sidelobe Ratio

- Compares the main peak height against the mean and spread of the *rest* of the correlation surface.
- A tall, isolated peak  $\rightarrow$  high PSR  $\rightarrow$  confident match.
- A flat, noisy surface  $\rightarrow$  low PSR  $\rightarrow$  reject it.
- Local threshold used: **PSR**  $>$  **10**.

*sharp, isolated peak*

$$\text{PSR} = \frac{g_{\max} - \mu_{sl}}{\sigma_{sl}}$$

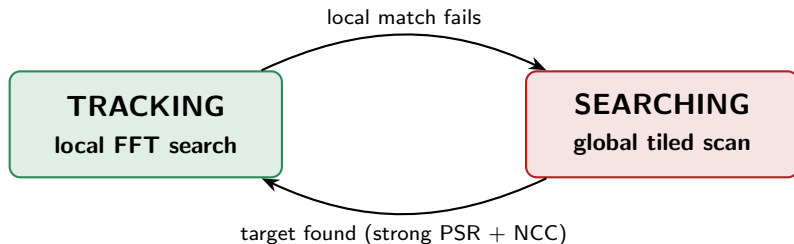
## NCC — Appearance Check

- PSR tells us whether the correlation peak is sharp.
- NCC tells us whether the matched patch actually looks like the stored target template.
- This prevents the tracker from trusting a high peak created by a structured background.
- Local threshold used:  **$NCC > 0.35$** .

### Simple meaning

NCC is a normalized similarity score from  $-1$  to  $1$ ; closer to  $1$  means stronger appearance match.

# Two-State Design



- Only **two** meaningful states — deliberately.
- “Occluded” is not a separate state: it is simply what *causes* a local match failure and the move into **SEARCHING**.

# Continuous Searching Instead of Final Failure

- The current design keeps the tracker in SEARCHING until the object is re-acquired.
- Reason: with a separate reference image, the selected object may appear much later in the video.
- So “not found yet” is treated as a normal searching condition, not a final failure.
- The system keeps scanning global tiles until a strong PSR/NCC match is found.

# The Solution: Time-Sliced Global Search

- Scanning the *whole* frame every video frame is too expensive with our custom (non-library) FFT.
- Instead, while SEARCHING:
  - the frame is divided into **overlapping** grid tiles,
  - only a **few tiles per video frame** are checked (default: 4),
  - this trades a little re-acquisition delay for stable, predictable CPU usage.
- Every tile is checked against the **fixed original reference template** (not the drifted working template) — so re-acquisition always anchors back to the true target.
- As soon as a tile passes strong PSR/NCC thresholds ( $\sim 15$  /  $\sim 0.50$ ), the Kalman filter is **reset** at that location and the system returns to TRACKING.

## Limitations — Stated Honestly

- **Translation-focused:** phase correlation is primarily built for translation, not scale/rotation.
- **Not scale-invariant:** a much bigger/smaller target than the template weakens matching.
- **Not rotation-invariant:** large rotation can break PSR/NCC.
- **Single target only:** tracks one selected identity at a time.

- **No semantic understanding:** it matches pixels, not the concept of “a car” or “a ball”.
- **Similar distractors:** a near-identical object nearby can pass PSR/NCC and cause a wrong lock.
- **Global search has latency:** a target visible only very briefly might vanish before its tile is reached.
- **CPU-bound:** the custom FFT is transparent and educational, but slower than optimized library/GPU FFTs.

- User can select a target from the video or provide a separate reference image.
- The app tracks the object locally using custom FFT phase correlation.
- It validates matches using PSR and NCC before updating the tracker.
- If the target is occluded, teleported, or appears later, global tiled search can re-acquire it.
- The Django interface shows the tracking result and the processing pipeline live.

# The Core Engineering Idea

Use a **cheap local tracker** while the motion assumption holds,  
switch to a **bounded global search** when it doesn't.

- Custom FFT + Phase Correlation
- PSR + NCC appearance validation
- Kalman motion prediction
- Time-sliced global grid search for re-acquisition
- Fixed reference template as an anchor
- Full DSP pipeline visualized live in a Django app

# Thank You

Questions?